

Supplementary Material

Accelerating a Regional Hyper-Resolution SWAT+ Model Without Changing the Science

Vahid Rafiei

This document provides reproducibility detail, the Fortran reentrancy hazard and its code, the full correctness forensics, and the extended scaling tables that support the main text. All measurements use the same benchmark model and protocol described in the main paper (Methods, Section 3). Section, figure, and table numbers prefixed with “S” refer to this document.

S1 Reproducibility: per-optimization commits and the build recipe

Every optimization in the cumulative ladder is a single commit in the engine fork (<https://github.com/rafiei-vahid/swatplus>), so the marginal contribution of each rung is auditable commit by commit (Table S1). The benchmark harness, the measured result files, the figure-generation script, and the benchmark model are released at <https://github.com/SWATGenX/swatplus-engine-acceleration>.

Table S1: The cumulative-ladder rungs, each a single commit. “Class” is the bottleneck the change addresses. Rungs R1/R2—the NetCDF output backend and the gauge print filter—are the I/O extensions documented and benchmarked in the companion paper [Rafiei, 2026]; they are available within R0 here and not re-derived, so the compute ladder begins at R3.

Rung	Commit	Optimization (class)
R0	768f1d1	Stock engine baseline (NetCDF/print-filter I/O backend available)
R3	87abf8e	<code>hru_read</code> $\mathcal{O}(1)$ name index (startup)
R4	247e95b	<code>varinit</code> per-row reset (per-step)
R5b	91922e3	Drop redundant inflow-hydrograph struct copy (channel)
R5c	6d41300	<code>ch_temp</code> reset scope (channel)
R6	34f5c02	Narrow-level fusion; final engine (serial + OpenMP builds)

Toolchain. The engine builds with the Intel oneAPI Fortran compiler (`ifx`) at `-O3 -ipo -recursive -init=zero -init=arrays -free -fpe0`; the OpenMP build adds `-fiopenmp` (`-DSWATPLUS_OPENMP=ON`). `gfortran` fails to produce a working binary at this model size.

Two build practicalities are easy to miss: a fresh CMake configure must pin `-DCMAKE_Fortran_COMPILER=ifx` and `PKG_CONFIG_PATH=<deps>/netcdf-ifx/lib/pkgconfig` (otherwise CMake selects `gfortran`/system NetCDF and the `.mod` files clash), and the Intel `setvars.sh` must be sourced without `set -u`. The `ifx` binaries link the Intel runtime and NetCDF (`libnetcdf.so.19`, `libnetcdf.so.7`, `libhdf5`); on a stock Ubuntu host these are `apt-installable`.

One binary, two modes. The final OpenMP binary runs full-wavefront by default and the byte-identical land-parallel/routing-serial mode when `SWATPLUS_ROUTING_SERIAL=1` is set in the environment (Section S3).

S2 The Fortran reentrancy hazard

Making the per-object body reentrant required auditing every module-global “current-object” scratch variable and giving each thread its own copy via OpenMP `threadprivate` (main text, Section 3.4). A Fortran-specific hazard complicates the naïve fix. A local declared with an initializer,

```
integer :: j = 0    ! implicitly SAVE in Fortran -> shared across calls
                  ! -> shared across threads -> a race
```

is *implicitly* `SAVEd`, hence shared across calls and therefore across threads. The reflexive fix—stripping the initializer so the local becomes automatic (per-thread under `-recursive`)—is correct *only* for write-before-read scratch. For the small set of accumulators that legitimately persist across calls, stripping the `SAVE` silently broke cross-call state and changed results *even at one thread*. The correct treatment is to keep those few variables `SAVEd` and mark them `threadprivate`, so each thread retains its own persistent copy. This was caught by the one-thread byte-identity check (Section S3): it surfaced a small ($\sim 0.3\%$) *deterministic* residual in `ch_watqual4` and `et_pot` where stripping `SAVE` had dropped persistent scratch; restoring it as `threadprivate/SAVE` state made one-thread output bit-identical again.

S3 Correctness forensics

Thread-count invariance as the acceptance test. For $N > 1$ we require repeated runs at different thread counts (1-vs-4, 4-vs-4, 4-vs-8) to produce identical daily output to a strict tripwire ($> 10^{-6}$ counts as a difference). Because a correct shared-memory program is deterministic regardless of scheduling, any run-to-run difference at a fixed thread count is, by definition, a data race, and the specific output fields that differ point at the unprotected state. We separate genuine races from benign arithmetic with two builds: a non-`ipo` build, bit-identical to the serial source (proving the algorithm is unchanged), and the `-ipo` build whose residual ($\sim 5 \times 10^{-4}$ relative) is floating-point reassociation.

The particulate in-stream race and its compiler root cause. A streamflow-only one-year test is insufficient: it passes while a latent race in the in-stream *particulate* constituents goes undetected. Only when in-stream water quality is exercised over a multi-year

run does thread-count invariance expose a divergence of order 15–20% in three species—organic-N, ammonia, and sediment-bound phosphorus—with run-to-run variation at higher thread counts confirming a race rather than reassociation. The HRU land phase is byte-identical across thread counts, localizing the race to the channel routing phase; the selectivity (particulate-only, dissolved NO_3 /soluble-P spared) localizes it further to the sediment-associated transport. We traced it with ThreadSanitizer (`ifx -fsanitize=thread`; the Archer-aware OpenMP runtime suppresses spurious synchronization reports). The race is *not* a missing `threadprivate`: every per-object index and scratch buffer is privatized. It is that `ifx` allocates the *temporaries* of the 152-byte `hyd_output` derived type (operator results, function returns, and argument copy-ins, used for every in-stream mass-balance expression) in **shared static module storage** rather than on the per-thread stack; concurrent channels then clobber the same static temporary mid-expression. The behavior is invariant to every compiler control we tried (`-recursive`, `-auto`, `-heap-arrays`, `elemental` operators, and disabling zero-initialization). A targeted source rewrite replacing the operator expressions with field-by-field helper subroutines is byte-identical at one thread but does *not* remove the race: passing a derived-type array element as a subroutine argument recreates a static copy-in temporary. A complete fix requires eliminating every derived-type temporary in every concurrently executed routine—inline, with no operators and no derived-type arguments—which is large and brittle, or a future compiler that stack-allocates these temporaries.

Resolution. Because the race lives entirely in the routing phase, running the land phase in parallel and the routing phase serially is race-free and byte-identical (run-to-run identical to $\sim 10^{-12}$ on basin constituent totals; thread-count-invariant to the floating-point-reassociation floor). This is the byte-identical mode of Table S3; the full wavefront—faster, bit-exact for flow/sediment/dissolved species—is the alternative mode for non-water-quality or tolerance-accepting use.

S4 Extended scaling tables

Table S2 gives the full cross-hardware scaling grid behind the main-text figure, including the stock baseline and serial gain per machine. Table S3 gives the full-wavefront versus byte-identical comparison on the reference node.

A note on the runs. The full-versus-byte-identical comparison (Table S3) is an *independent* run on the reference c8a node, with its own stock baseline $R_0 = 193.5\text{ s}$ —within 0.8% of the cross-hardware run’s 191.9 s (Table S2). The three slightly different one-thread parallel walls (143.1 s in Table S2; 142.9 and 143.5 s here) are these independent best-of-two measurements, not a discrepancy.

Composing the layers. The end-to-end speedup is the product of three independent layers: output scope (an I/O effect, quantified for these models in the companion benchmark [Rafiei, 2026] and excluded from the main figures), the machine-portable serial gain ($1.67\text{--}2.47\times$), and the thread scaling above. Against the comparison run’s $R_0 = 193.5\text{ s}$ the directly-

Table S2: Cross-hardware scaling of the final engine (each machine’s own $1 \rightarrow N$ ratio), identical engine, one-simulated-year benchmark workload, daily `channel_sd` output, thread-pinned, best of two, on dedicated AWS instances.

	c8a (Turin)	c8i (Granite Rapids)	c5a (Rome)
physical cores (vCPU)	32 (32)	16 (32)	16 (32)
SMT	off	on (2 $\times$)	on (2 $\times$)
stock R_0 (s)	191.9	272.2	350.6
serial gain	1.67 $\times$	2.47 $\times$	2.16 $\times$
t_1 (s)	143.1	130.1	202.5
2	1.70 $\times$	1.59 $\times$	1.62 $\times$
4	2.79 $\times$	2.41 $\times$	2.47 $\times$
6	3.48 $\times$	2.93 $\times$	2.89 $\times$
8	3.95 $\times$	3.29 $\times$	3.21 $\times$
10	4.30 $\times$	3.55 $\times$	3.42 $\times$
12	4.54 $\times$	3.75 $\times$	3.60 $\times$
16	4.90 $\times$	4.01$\times$	3.81$\times$
20	5.18 $\times$	3.73 $\times$	3.51 $\times$
24	5.33$\times$	3.74 $\times$	3.58 $\times$

measured stock $\rightarrow$ parallel total is 3.78 $\times$ (full) / 2.75 $\times$ (byte-identical) at four threads and 5.39 $\times$ / 3.37 $\times$ at eight; on the cross-hardware run ($R_0 = 191.9$ s) the full wavefront reaches 7.14 $\times$ at 24 threads.

References

Vahid Rafiei. Automating high-resolution SWAT+ watershed model generation from NHDPlus HR and national environmental datasets. *Computers & Geosciences*, 2026. Manuscript submitted for publication; *Computers & Geosciences* CAGEO-D-26-00981.

Table S3: Full wavefront vs. byte-identical (routing-serial) mode on a dedicated c8a instance, same final engine. Self-relative speedup and the forfeit (the parallel speedup the byte-identical mode gives up by serializing routing).

Threads	Full wavefront		Byte-identical		Forfeit
1	142.9 s	1.00×	143.5 s	1.00×	—
2	84.2 s	1.70×	97.9 s	1.47×	14%
4	51.2 s	2.79×	70.3 s	2.04×	27%
8	35.9 s	3.98×	57.4 s	2.50×	37%
16	29.3 s	4.88×	51.0 s	2.81×	43%